INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN

Projet intégratif développement orienté objet

Réalisé par :

Arrigoni Ambroise

Belkouchi Chahd

Yomni Anass

Douagi Selma

Année universitaire 2024/2025

Table des matières

1 Introduction **3**

2 Persistance BD **4**
2.1 Passage E/A → Modèle relationnel 4
2.1.1 Étapes générales 4
2.1.2 Cas 1 : Entité non-faible 4
2.1.3 Cas 2 : Entité faible 4
2.1.4 Cas 3 : Associations $\langle 1, 1 \rangle$ 4
2.1.5 Cas 4 : Autres associations 5
2.2 Choix réalisés pour le modèle physique 5
2.3 Implémentation d'EnregistreurBD 5
2.4 Implémentation du LecteurBD 6
2.5 Amélioration possible 6

3 Interprétation fichier structuré **7**
3.1 Pourquoi avons-nous besoin de JavaCC et d'appliquer nos cours de compilation ? 7
3.2 Patron de conception Visiteur/Visitable 7
3.2.1 Analyse Syntaxique 7
3.2.2 Analyse Sémantique 8
3.2.3 Phase de Traitement 9

4 Design patterns **10**
4.1 Introduction 10
4.2 ITIAventureFactory 10
4.2.1 Lecteur Description 10
4.3 ITISpaceOpera 11

Chapitre 1

Introduction

L'objectif de ce projet s'inscrit dans la continuité du projet ITI Aventure, réalisé dans le cadre de l'EC *Programmation Avancée en Java*. Ce dernier consistait à créer un jeu d'aventure au tour par tour. Le jeu est composé d'un monde contenant des entités de structuration spatiale (Pièce, Porte, Objet...) et d'entités vivantes interagissant dans ce monde (Monstre, Humain...).

Dans le cadre de ce nouveau projet, il s'agit de restructurer l'existant et d'ajouter des fonctionnalités au jeu :

- Mise en place de design patterns (Singleton, Factory, MVC...) : la modélisation objet actuelle du projet est perfectible. Il existe des patrons de conception classiques (de création, de structure et de comportement) qu'il est pertinent d'appliquer. Certains de ces patterns devront être implémentés.

- Persistance en base de données relationnelle : la sérialisation des objets se fait actuellement dans un fichier. L'objectif de cette partie est de gérer la persistance des objets dans une base de données relationnelle.

- Lecture d'un fichier de configuration structuré : le format actuel du fichier est limité et peu robuste face aux erreurs. Un nouveau formalisme est proposé, et il s'agira de mettre en œuvre l'interprétation de ce nouveau format.

Les objectifs pédagogiques liés à ce projet sont les suivants :

- Choisir la structure de données et le traitement approprié selon la nature du problème. - Identifier les flux de données et sélectionner les traitements les mieux adaptés, après avoir prototypé différentes solutions. - Structurer les données et leurs flux, puis créer ou adapter les traitements associés. - Développer un logiciel autonome en langage orienté objet, en utilisant des bibliothèques adaptées, au sein d'une équipe de 4 à 5 personnes. - Développer un logiciel client-serveur en équipe, en utilisant des frameworks spécifiques. - Gérer le développement et la qualité du code tout au long du projet.

Chapitre 2

Persistance BD

Au cours de notre projet, nous devions implémenter un modèle de base de données conforme au modèle conceptuel imposé sur Moodle.

2.1 Passage E/A → Modèle relationnel

2.1.1 Étapes générales

- **Cas 1 :** Entité non-faible
- **Cas 2 :** Entité faible
- **Cas 3 :** Associations $\langle 1, 1 \rangle$
- **Cas 4 :** Autres associations

2.1.2 Cas 1 : Entité non-faible

- `Porte(nom_porte, état)`
- `Monde(nom_monde)`
- `JoueurHumain(nom_joueur, ptsVie, ptsForce)`
- `PiedDeBiche(nom_pdb, estDéplaçable)`
- `Pièce(nom_piece)`

2.1.3 Cas 2 : Entité faible

Aucune entité faible dans le modèle imposé.

2.1.4 Cas 3 : Associations $\langle 1, 1 \rangle$

Types d'associations
- Contient (Monde–Porte)
- Contient (Monde–Pièce)
- Contient (Monde–JoueurHumain)
- Contient (Pièce–JoueurHumain)

Relations générées
- `Porte(nom_porte, état, nom_monde)`
- `Monde(nom_monde)`
- `JoueurHumain(nom_joueur, ptsVie, ptsForce, nom_monde, nom_piece)`
- `PiedDeBiche(nom_pdb, estDéplaçable)`
- `Pièce(nom_piece, nom_monde)`

2.1.5 Cas 4 : Autres associations

- Possède_Piece_Porte(nom_porte, nom_piece, nom_monde)
- Contient_Piece_Pdb(nom_piece, nom_pdb, nom_monde)
- Possède_Pdb_JoueurHumain(nom_pdb, nom_joueur, nom_monde, nom_piece)

2.2 Choix réalisés pour le modèle physique

Pour faciliter l'implémentation du modèle physique tout en respectant les contraintes du projet, nous avons effectué plusieurs adaptations.

Ajout d'une clé étrangère dans la table `PiedDeBiche` : Nous avons ajouté une clé étrangère `nom_monde` à la table `PiedDeBiche`, ce qui donne la structure suivante :

PiedDeBiche(nom_pdb, estDéplaçable, nom_monde)

Cette clé permet, lors de la lecture de la base de données, d'identifier le monde auquel appartient un pied de biche et ainsi de retrouver précisément l'objet concerné.

Modification de l'association `Possède_Piece_Porte` : Nous avons modifié la table représentant l'association `Possède_Piece_Porte` pour y inclure directement les deux pièces reliées par une porte. La nouvelle structure est la suivante :

Possède_Piece_Porte(nom_pieceA, nom_pieceB, nom_porte)

Cette simplification permet une lecture plus directe de la base de données, en rendant explicite la relation entre les deux pièces et la porte qui les relis.

2.3 Implémentation d'EnregistreurBD

Pour l'implémentation de l'enregistreur, nous avons utilisé un `PreparedStatement` distinct pour chaque table SQL. Cela nous permet d'utiliser des requêtes SQL précompilées, ce qui améliore la performance de l'insertion des données.

Nous avons ensuite créé deux fonctions pour chaque type d'entité :

- `creerEntite` : ajoute une instance de l'entité à la base de données.
- `creerLesEntites` : appelle `creerEntite` pour ajouter toutes les entités d'un certain type.

Ordre d'insertion des entités Pour garantir la cohérence de la base de données, nous avons réfléchi à l'ordre dans lequel les entités devaient être insérées.

Logiquement, nous commençons par insérer **le monde**, car il contient la clé primaire utilisée comme clé étrangère dans les autres tables.

Ensuite, nous insérons les **pièces**, avant les **portes**. Pourquoi ce choix ? Car les portes sont contenues dans les pièces (chaque porte appartient à une pièce), il est donc nécessaire que les pièces soient déjà présentes dans la base.

Enfin, nous ajoutons les autres entités comme les joueurs humains et les objets (par exemple les pieds-de-biche).

Extraction des entités : portes, pièces, joueurs humains, pieds-de-biche
Nous avions deux options pour extraire ces entités :

— Ajouter des getters spécifiques dans la classe `Monde`.

— Utiliser les *Streams* Java avec les méthodes `filter` et `map`.

Ne sachant pas si nous étions autorisés à modifier `Monde.java`, nous avons choisi d'utiliser le getter existant `monde.getEntites()` et de filtrer les entités via les *Streams* :

```
List<Piece> pieces = entites.stream().filter(e -> e instanceof Piece)
                            .map(e -> (Piece) e)
                            .collect(Collectors.toList());

List<JoueurHumain> jhs = entites.stream().filter(e -> e instanceof JoueurHumain)
                                .map(e -> (JoueurHumain) e)
                                .collect(Collectors.toList());
```

Cependant, cette méthode implique de parcourir l'ensemble des entités à chaque fois, ce qui peut être inefficace. Une amélioration possible serait de *parcourir la liste une seule fois*, et de répartir les entités dans plusieurs listes selon leur type. Cela permettrait de réduire la complexité de l'opération et d'optimiser les performances.

2.4 Implémentation du LecteurBD

L'implémentation nous a emmené à apporté les corrections nécessaires à la base de données que nous avons cités dans la section 2.2 . Ci-dessous, un exemple d'une table de la base de données :

```
grtt1=> select * from monde;
          nom
------------------------
 LeMondeImpitoyabledITI
(1 row)

grtt1=> select * from porte;
      nom       |       nom_monde        |    etat
----------------+------------------------+------------
 Porte5         | LeMondeImpitoyabledITI | FERME
 Porte4         | LeMondeImpitoyabledITI | FERME
 PassageSecret  | LeMondeImpitoyabledITI | FERME
 Trape          | LeMondeImpitoyabledITI | FERME
 Porte1         | LeMondeImpitoyabledITI | VERROUILLE
 Porte3         | LeMondeImpitoyabledITI | FERME
 Porte2         | LeMondeImpitoyabledITI | VERROUILLE
(7 rows)

grtt1=>
```

2.5 Amélioration possible

Dans la partie base de données, nous aurions pu ajouter un attribut `ID` pour remplacer le rôle de clé étrangère actuellement joué par `nom_monde`. En effet, le nom du monde étant une chaîne de caractères, il est dupliqué de nombreuses fois dans les différentes tables. Or, une chaîne de caractères occupe bien plus d'espace mémoire qu'un simple identifiant numérique. L'utilisation d'un `ID` permettrait donc d'optimiser l'espace mémoire et d'améliorer les performances des requêtes.

Chapitre 3

Interprétation fichier structuré

Au cours de notre projet, nous devions également assurer la lecture d'un fichier de configuration structuré : le fichier actuel étant limité et sans gestion des erreurs, un nouveau formalisme est proposé.

3.1 Pourquoi avons-nous besoin de JavaCC et d'appliquer nos cours de compilation ?

Le fichier à notre disposition montre que nous travaillons sur un langage algébrique. En effet, le langage est équilibré, mais il ne s'agit pas simplement d'un langage rationnel, comme on peut le constater à la ligne 7 avec `"serrure(m1))"`. Ainsi, un compilateur est nécessaire, et l'utilisation de JavaCC facilite la conception de l'analyseur lexical et syntaxique.

3.2 Patron de conception Visiteur/Visitable

Afin de séparer les algorithmes de traitement des données, nous avons implémenté le patron de conception Visiteur/Visitable. Nous avons fait le choix de créer une classe pour chaque nœud de l'AST. Ainsi, nous avons appliqué le modèle originel du patron Visiteur, pour sa simplicité et en raison du temps limité dont nous disposions.

3.2.1 Analyse Syntaxique

L'analyse syntaxique commence après la phase lexicale (générée aussi par JavaCC), qui découpe le fichier en token comme :

— des noms de fonction (`monde`, `piece`, `porte`),
— des identifiants (`m1`, `bureau_nicos`),
— des chaînes (`"BureauDesNicolas"`, `"Porte1"`),
— des entiers,
— des états du jeu (`SUCCES`, `ECHEC`, `ENCOURS`),
— des symboles : parenthèses, virgules et signe égal (`=`).

Évolution du design

Au début, nous avions fait le choix d'utiliser une seule classe `Argument` pour englober tous les type possibles (entier, chaîne, nom de fonction, identifiant, appel de fonction), avec un attribut interne `type` pour indiquer la catégorie. Ce design paraissait plus compact et facile à écrire, mais il rendait le code moins claire et lisible, et s'est avéré complexe à manipuler lors des phases ultérieures au niveau du transtypage par exemple.

Nous avons donc décidé de changer d'approche et d'opter pour une classe dédiée par type d'argument : `Entier`, `Chaine`, `Identifiant`, `NomFonction`, `AppelFonction`, etc. Ce choix s'est avéré plus simple, plus robuste, et plus lisible : chaque classe sait ce qu'elle représente et peut être directement visitée sans vérification manuelle du type.

Règles syntaxiques

— **Affectation** := Identifiant `=` AppelFonction
— **AppelFonction** := NomFonction `(` ListeArguments `)`
— **ListeArguments** := Argument+
— **Argument** := Entier | EtatDuJeu | Chaine | Identifiant | AppelFonction

Prenons un exemple avec la première règle, qui indique qu'une ligne doit correspondre à une affectation d'un identifiant à un appel de fonction.

Exemple : porte1 = porte(m1, "Porte1", bureau_nicos, couloir, serrure(m1))
Décomposition :

— une affectation : `porte1 = porte(...)`,
— un appel de fonction : `porte(...)`,
— une liste d'arguments composée d'identifiant, d'une chaîne et d'un appel de fonction imbriqué.

Si la structure respecte les règles, l'analyseur génère un AST composé :

— d'un objet `Confuguration` qui contients toutes les affectations
— des instances de la classe `Affectation`,
— des instances de la classe `AppelFonction`,
— et des instances des classe comme `Entier`, `Chaine`, `Identifiant`, `EtatJeu` selon le type exact de chaque argument.

Ce découpage fin, combiné à notre changement de design, a considérablement simplifié l'écriture de l'analyse sémantique et de la phase de traitement.

3.2.2 Analyse Sémantique

Pour garantir la cohérence et la validité de notre fichier de configuration, nous avons décidé d'implémenter une analyse sémantique, en complément des phases lexicale et syntaxique.

Les vérifications effectuées incluent notamment :

— la vérification de l'existence préalable des identifiants utilisés,
— le contrôle du bon type des arguments passés aux fonctions,
— la détection des redéfinitions interdites.

Nous avons fait le choix d'implémenter l'analyse sémantique directement dans les classes `AppelFonction` et `LecteurCompil`, principalement pour des raisons de contraintes temporelles.

Plusieurs éléments ont renforcé ce choix :

— **Simplicité d'intégration** : en regroupant les vérifications sémantiques proches des structures manipulées, nous avons évité de multiplier les classes ou modules séparés, ce qui a simplifié l'architecture générale et limité les risques d'erreurs liés à la complexité.
— **Adaptation aux besoins du projet** : l'objectif principal était d'assurer une validation fonctionnelle du projet dans les délais, sans nécessairement chercher à construire une architecture entièrement découplée dès la première version.
— **Cohérence locale** : certaines vérifications (comme le contrôle des paramètres dans un appel de fonction) sont étroitement liées à la logique de la classe `AppelFonction`, et les déplacer dans un module séparé aurait nécessité de créer des mécanismes supplémentaires de communication entre les classes.

3.2.3 Phase de Traitement

La phase de traitement a été implémenté directement dans la classe `LecteurCompil`, qui joue le rôle de **visiteur** pour les différentes classes `visitable` de l'AST : `Configuration`, `Affectation` et `AppelFonction`.

Concrètement, nous avons défini trois méthodes principales :
— `visiter(Configuration)` : cette méthode récupère d'abord toutes les affectations présente dans la configuration, puis, pour chacune, appelle `accepter()` pour déclencher le traitement par le visiteur.
— `visiter(Affectation)` : cette méthode récupère l'appel de fonction associé à l'affectation, puis appelle à son tour `accepter()` sur l'appel de fonction, afin de poursuivre le traitement.
— `visiter(AppelFonction)` : cette méthode prépare la liste des arguments nécessaires pour créer l'objet correspondant dans le monde du jeu au fur et à mesure.

Nous avons choisi cette architecture pour plusieurs raisons :
— **Centralisation** : en regroupant la logique de traitement dans `LecteurCompil`, nous évitons la dispersion du code et facilitons le suivi du processus global.
— **Séparation des rôles** : chaque classe `visitable` (comme `AppelFonction` ou `Affectation`) délègue son traitement au visiteur via l'appel à `accepter()`, ce qui respecte le patron de conception Visiteur et permet de séparer les données de leur traitement.
— **Facilité de réutilisation** : cette organisation nous permet, si nécessaire, d'ajouter de nouveaux comportements ou de nouveaux types visitables sans modifier en profondeur les classes existantes.

Ainsi, chaque élément de l'AST sait simplement comment appeler `accepter()` pour laisser le `LecteurCompil` exécuter la bonne méthode `visiter()`, assurant un traitement clair, structuré et extensible.

Chapitre 4

Design patterns

4.1 Introduction

Afin d'améliorer et optimiser notre code du projet ITIAventure, et donc notre projet integratif orienté objet, on souhaite introduire une partie uml et design patterns.

Cette partie a pour but de découpler le code de création de certaines entités du monde ITIAventure.

4.2 ITIAventureFactory

Pour decentraliser notre code, nous avons choisi d'implémenter une abstract factory dans le package fr.insarouen.iti.prog.aventure. qu'on a nommé ITIAventureFactory qui contient les méthodes abstract qui devront être complétée dans les classes qui héritent de cette factory et qui devront créer les classes Monde, Monstre, Piece, Porte, Serrure et Cle.
On a ensuite créé la classe ITIAventureFactoryCreation, c'est la classe qui hérite de ITIAventureFactory et redéfinit ses méthodes abstract, ces méthodes consistent généralement à retourner une instance de la classe en question en fonction des paramètres.

- Porte : Pour créer une porte on vérifie d'abort si la serrure a été entrée en paramètres, on utilise le constructeur avec serrure le cas échéant, sinon, on utilise l'autre constructeur sans Serrure ;
- Cle : la clé est instanciée à partir de la serrure ;
- Serrure : pour la serrure on a choisit de l'instancier à partir du constructeur qui ne prend en paramètres que le monde.

4.2.1 Lecteur Description

Pour utiliser les factorys créées on modifie notre classe LecteurDescription dans le package fr.insarouen.iti.prog.aventure.data. On a modifié le constructeur de la classe de facon à ce qui'il prenne en considération le nouveau fichier de simulation

dans lequel le nom de la factory à utiliser a été ajouteé dans la première ligne. Après avoir lu cette ligne, on instancie une factory avec le nom lu. Dans les méthodes de création, au lieu d'instancier directement de la classe, on utilise la factory.

4.3 ITISpaceOpera

ITISpaceOpera est une aventure futuriste qui s'appuie sur ITIAventure avec les correspondance s suivantes :

- Monde Galaxie
- Piece : VaisseauSpatial
- Porte : Teleporteur
- Cle : Badge
- Serrure : LecteurBadge
- Monstre : Alien.

On a créé les classes correspondantes à ces entités dans le package fr.insarouen.iti.prog.aventure.spa Chacune de ces classes hérite de la classe qui lui correspond, et on utilise tout simplement un super dans le constructeur.

Dans la classe LecteurBadge, qui correspond à la serrure, on a redéfinit la méthode creerCle() pour que le badge (Correspondant à la clé) soit instancié à partir du LecteurBadge.

Dans l'implémentation de cette méthode, on utilise le nom de la clé nomCle, or cet attribut était de visibilité private ; on a choisit donc de le mettre en protected pour pouvoir y accéder dans la méthode de la classe fille.

On a créé ensuite la classe ITISpaceOperaFactory qui hérite elle aussi de la ITIAventureFactory. Elle consiste sur le même principe de ITIAventureFactoryCreation et retourne des instances des classes nouvellement créées de ITISpaceOpera ;

Ayant créé cette nouvelle aventure, nous devons remodifier la classe LecteurDescription pour qu'elle prenne en considération la nouvelle factory.

On a gardé la même logique, c'est-à-dire qu'on lit d'abord le nom de la factory puis on utilise un switch-case :

- si le nom lu est "ITISpaceOpera", on utilise une instance de la ITISpaceFactory
- si le nom lu est "ITIAventure", on utilise une instance de ITIAventureFactoryCreation
- sinon, on lance une exception.

Conclusion

Ce projet nous a permis de consolider nos compétences en programmation orientée objet et en architecture logicielle avancée à travers la réorganisation et l'extension d'un jeu d'aventure existant. Nous avons appliqué plusieurs concepts étudiés en cours, notamment les patrons de conception (Factory, Singleton, Visiteur/Visitable), la persistance des données via une base relationnelle, ainsi que la lecture et l'interprétation d'un langage de configuration structuré en utilisant JavaCC.

Ainsi, nous avons mis en œuvre un modèle physique de base de données cohérent avec le modèle E/A qui était fourni. L'utilisation du patron Visiteur a facilité l'analyse syntaxique du fichier de configuration, tout en maintenant une séparation claire entre les données et les traitements. Par ailleurs, la mise en place d'une architecture extensible via une factory abstraite nous a permis d'introduire facilement un nouvel univers de jeu (ITISpaceOpera), illustrant la flexibilité de notre conception.

De plus, malgré certaines contraintes de temps, nous avons fait des choix techniques pertinents et identifié des pistes d'amélioration (comme l'introduction d'identifiants numériques dans la base de données ou l'optimisation des filtres sur les entités).

Finalement, ce projet a été une expérience enrichissante, à la fois technique et collaborative, qui nous a permis d'appliquer concrètement les notions de conception logicielle, de compilation, et gestion de données dans un contexte complet et stimulant.

Tâche	**Ambroise**	**Selma**	**Chahd**	**Aness**
Persistance BD	X	X		
Interprétation Fichier structuré	X	X		
Design patterns			X	X
Rapport	X	X	X	X